

PolaGrid — Kick-off Q&A

Hyunseok Lee · Sejong University, Intelligent IoT Convergence · 1-person team · oneM2M + MEC Hackathon 2026

Q1. What is PolaGrid?

We cut the window into small cells. Each cell has two polarizing films. A servo motor turns one film. When the sun shines too hard on your desk, only the cells that let that sun in get darker. The other cells stay clear, so the room stays bright and you can still see outside.

Q2. Why use polarizing films and not window blinds?

Blinds cover a big part of the window at once. With two polarizing films, turning one film lets in more or less light, step by step. So we can make just one cell darker. Weak point: the films take away some light even when fully open, and they work best on light that bounces off a desk or screen, less on direct sun.

Q3. The video says "AI". What does the AI do?

Small, simple AI models, no camera, only light sensors. (1) Right after setup, it darkens one cell at a time and watches the desk light sensor, so it learns which cell sends light to which spot. (2) It picks the angle of each cell to cut the sun on the desk while keeping the desk bright enough. (3) It learns from past days and from the user's own changes in the app, and guesses good angles for the next day.

Q4. How do you use oneM2M and MEC?

Each area of a building has its own small server (tinyIoT MN-CSE) close to the windows, on an edge (MEC) machine. One main server (tinyIoT IN-CSE) sits above them. Each cell saves its light value, angle and command there. One message to a group reaches all cells in the area, and each cell gets told when a new command arrives. The control program runs as a MEC app. Until we get the contest MEC setup, we use a fake local one.

Q5. Can it work for many windows?

We use real cells (2, then a 12-cell window) and many computer-made cells (100, then 1,000 and 10,000), all on the same oneM2M server tree. First test on one laptop: with 1,000 cells (10 servers x 100 cells), a command reached all cells within 382 ms (95% of the time) from the nearby server, and 801 ms through the main server. No message was lost out of 10,000. Our goal is under 5 seconds.

Q6. Why tinyIoT and not ACME or Mobius?

In our test, ACME could save at most 78 requests per second. tinyIoT, after a small fix we made to how it saves data, did 577 per second on one server. We keep Mobius as a second main server to show that different oneM2M servers can work together.

Q7. How do you show it works better?

We play back the same weather and compare with three simple ways: always open, close the whole area when it is too bright (like blinds), and change on a fixed time table. We count: hours with too much sun on the desk, desk brightness, hours too dark to work, how often servo motors move, and command time. We will not claim energy savings we did not measure.

Q8. What do you get from the platform besides control?

The same data can be used by other apps. A dashboard and a separate building app read the same data from the server, without talking to the device directly.

Q9. What is already done?

A working model (light sensor + servo motor + rotating polarizing film) from a 2025 class project, a test setup with 1 main server and 100 area servers, test tools, and 3 bug fixes for tinyIoT.

Q10. You work alone. Can you finish?

The plan has 3 steps: AI on 2 real cells, then one real window with 12 cells, then many computer-made cells on oneM2M. If time runs out, I drop the extra dashboard work and the 10,000-cell test first. The code will be open (Apache 2.0) with a short setup guide.

Q11. What problems are left?

The server gets slower when a lot of old data piles up (I am testing this over long runs). The area servers are not yet on separate machines. The MEC setup depends on the contest team.